Akka, it's easier because you manage state sequentially, message by message, within the actor. Let's see how.

State can be implemented by behaviors: that is, functions. In this example, the wallet stores a value in `total` that can increase or decrease within certain limits. To keep things simple, the maximum value is set to `2` and the minimum value is set to `0` (zero). When the maximum/minimum values are exceeded, the wallet stops increasing/decreasing and the actor stops running, signaling that the wallet is in an unacceptable state.

You already know what a protocol is. But actors with integers as their protocol are used only for example purposes. Any real use-case protocol is implemented with several types belonging to a hierarchy with a sealed *trait* at the top.

DEFINITIONS *Traits* are like interfaces in Java. For more information, go to https://docs.scala-lang.org/tour/traits.html. A *sealed trait* is a trait that allows the compiler to check whether your pattern matching over its cases is exhaustive. When it is not exhaustive, the compiler raises a warning. For more information, go to the section "Sealed types" at http://mng.bz/Pz8v.

Instead of using `Behavior[Int]`, as in the previous wallet, this wallet has `Behavior [Command]`. Its protocol has two types of messages, `Increase(amount: Int)` and `Decrease(amount: Int)`, both belonging to the `Command` trait:

```
object WalletState {

  sealed trait Command
```